

Assignment#01: With vibe coding implement the playwright/typescript or any other tool which you work say Selenium/Python Sample prompt for playwright/typescript integration: Ask mode : Read my entire workspace and confirm me what are the files will be impacted if I want to add playwright typescript same like implemented for Selenium Java.

Given this Prompt in Co-Pilot

1. coding implements the playwright with Typescript

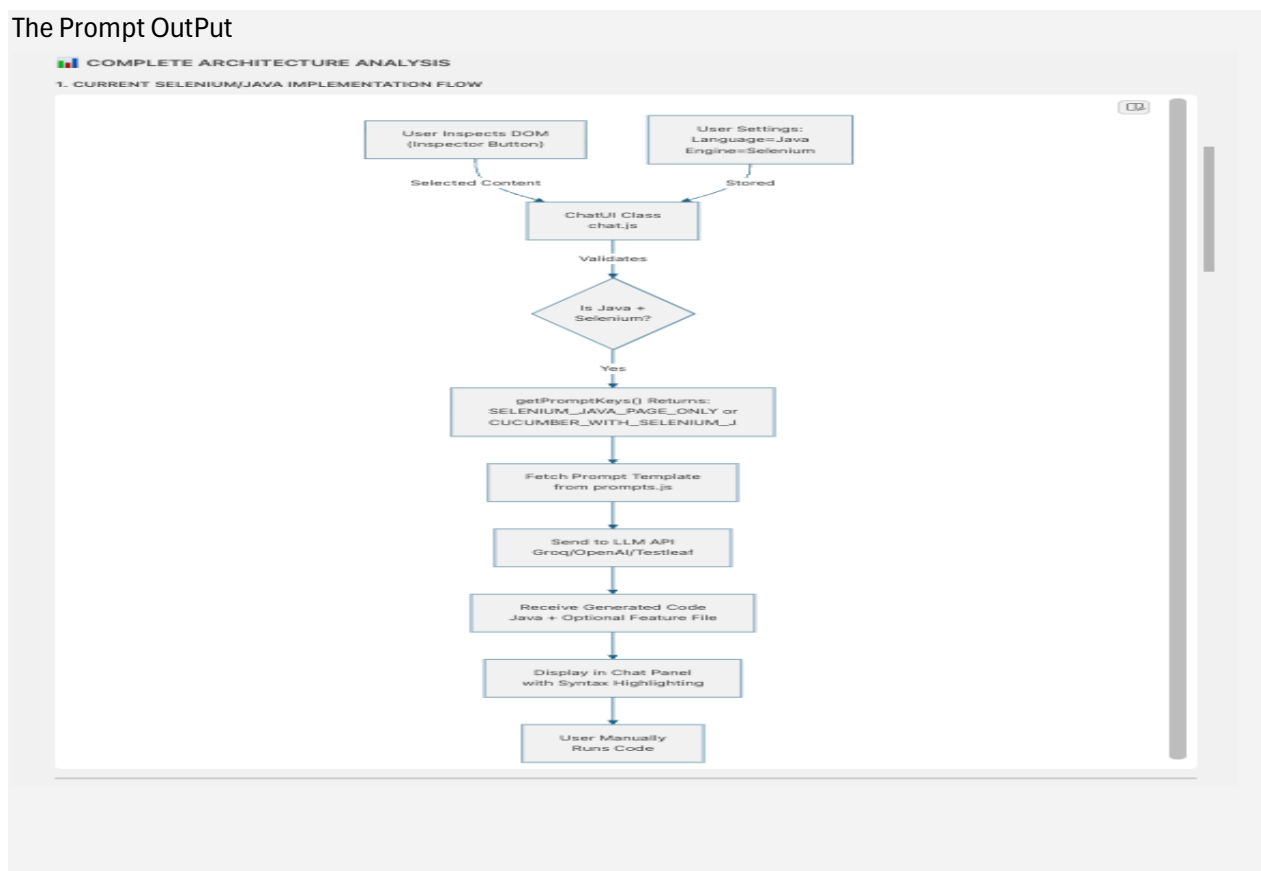
Custom Prompt

I have selected

given below Prompt

- Analyze my entire code base and give the impacted files.
- Existing Selenium Java implementation flow.
- Also the required Architecure changes to implement.
- Playwright with typescript step by step with the impacted files like implemented with Selenium with Java

The Prompt OutPut



2. IMPACTED FILES - DETAILED BREAKDOWN

2. IMPACTED FILES - DETAILED BREAKDOWN

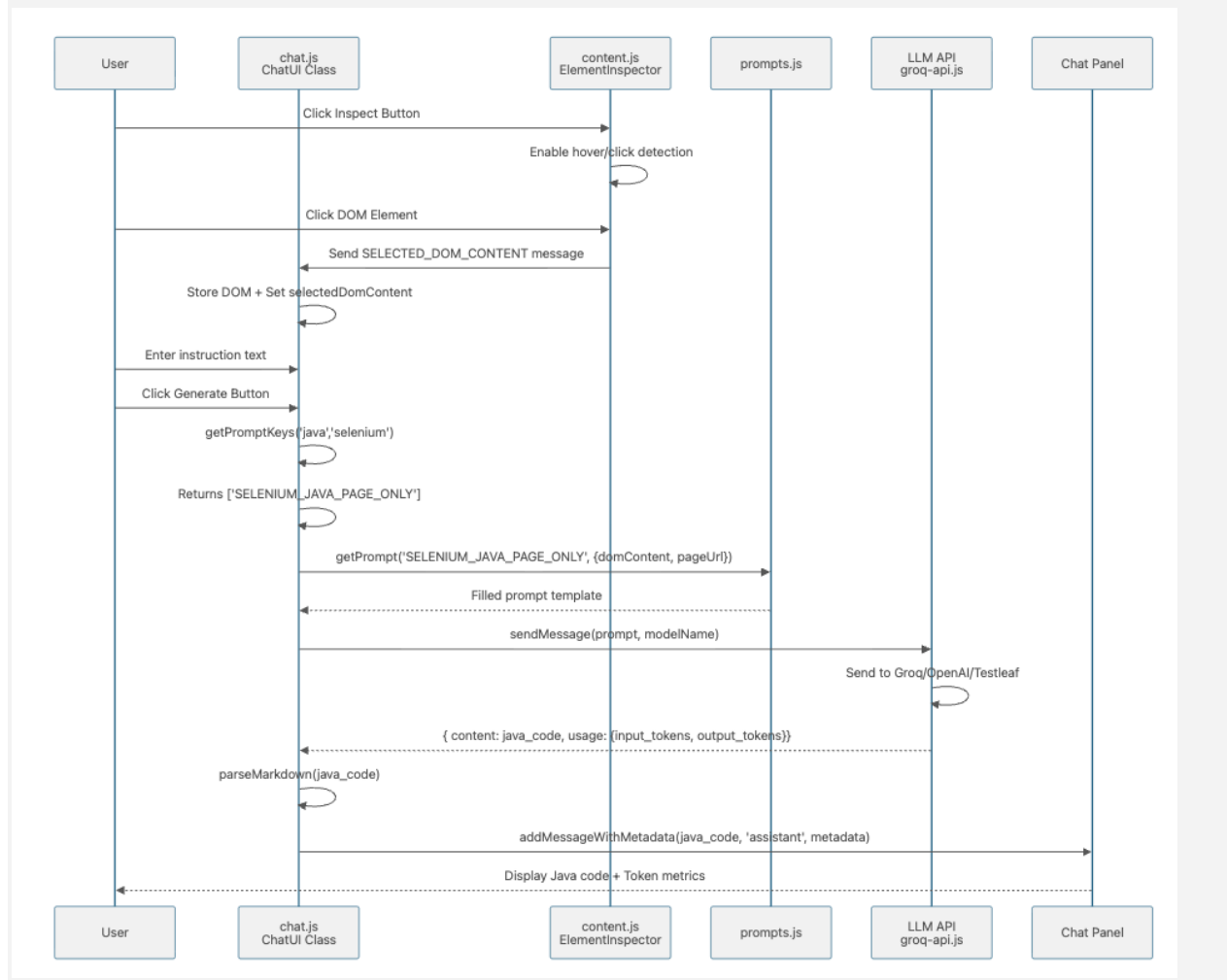
FILES WITH CHANGES NEEDED:

File	Changes	Priority	Impact
chat.js	Add Playwright logic to <code>getPromptKeys()</code> ; Already has methods but needs execution handler	HIGH	Core logic for routing
<code>prompts.js</code>	✓ ALREADY HAS Playwright prompts (POM, SPEC, POM_AND_SPEC)	MEDIUM	Just verify & use
<code>panel.html</code>	Add "Run Tests" button for Playwright; Conditional visibility	MEDIUM	UI/UX improvement
<code>popup.js</code>	Minor: Ensure settings work for TypeScript language	LOW	Settings management
playwright-api.js	CREATE NEW - Execute tests locally	CRITICAL	Test execution engine
project-runner.js	CREATE NEW - Setup project structure	CRITICAL	Project scaffolding

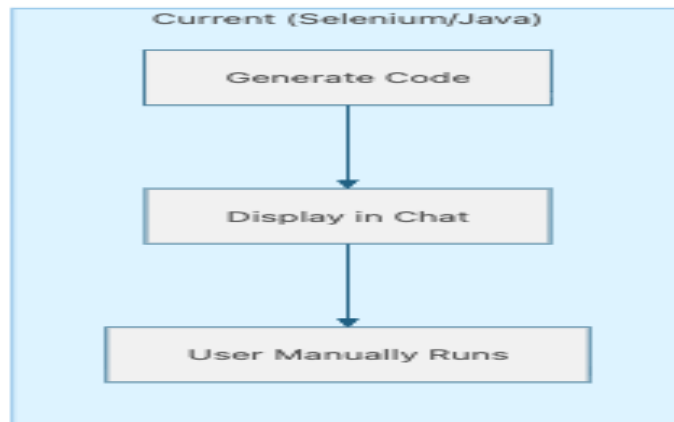
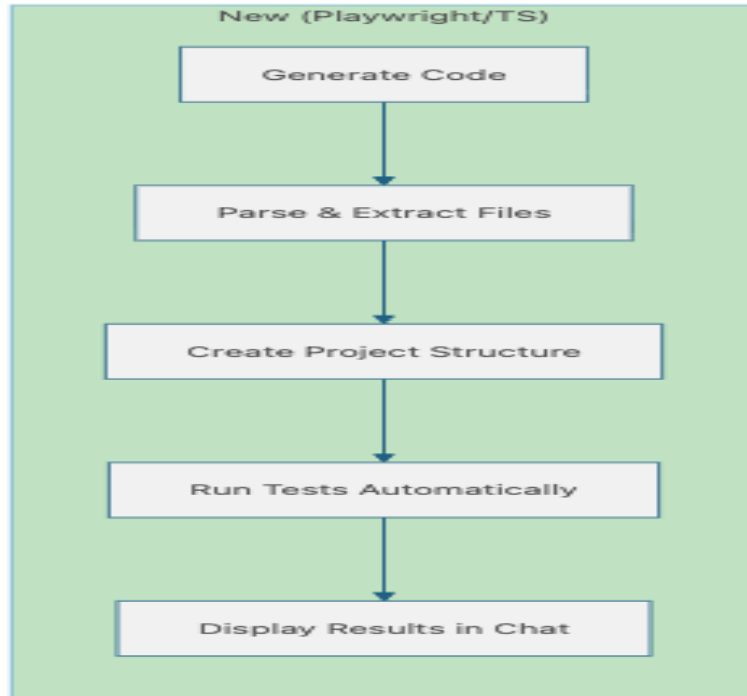
FILES WITH NO CHANGES:

- `bg.js`, `content.js`, `configUtils.js`, `init-colors.js` (language-agnostic)

3. EXISTING SELENIUM/JAVA FLOW (DETAILED)



4. REQUIRED ARCHITECTURE CHANGES



5. User -> Agent

Existing supports selenium Java Add new changes to support Playwright with Typescript Both Selenium Java and SPlaywright with Typescript should be supported No impact in the existing functionality.



After Click on Enter

Existing supports selenium Java Add new changes to support Playwright with Typescript Both Selenium Java and SPlaywright with Typescript should be supported No impact in the existing functionality.

Optimized tool selection

Planned Playwright implementation with TypeScript support

I'll implement Playwright with TypeScript support while maintaining full backward compatibility with Selenium/Java. Let me start by creating a detailed implementation plan.

Created 7 todos and reviewed prompts.js

Now let me start implementing. I'll mark the first task as in-progress and create the necessary files.

Created PlaywrightAPI class for test execution

Now I'll create the ProjectRunner utility class:

Starting: Create ProjectRunner class for project setup (2/7)

- Loading

>  Create ProjectRunner class for project setup (2/7)

> 1 file changed +206 -1

Keep

Undo



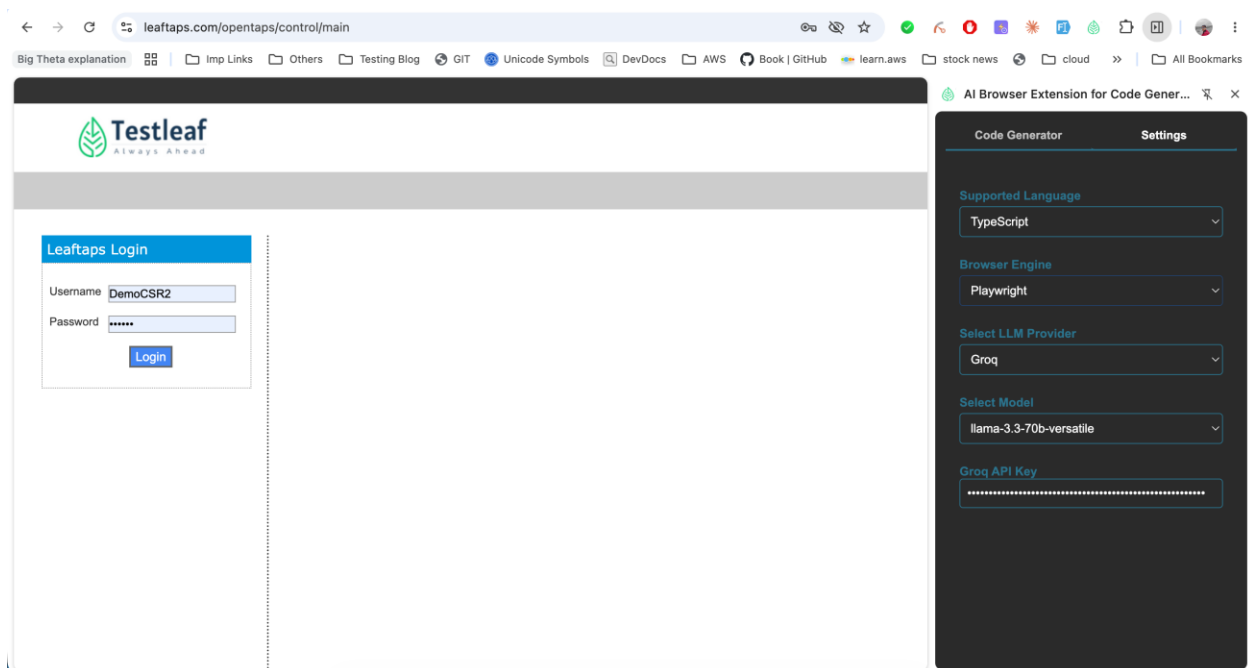
+ JS chat.js

Describe what to build

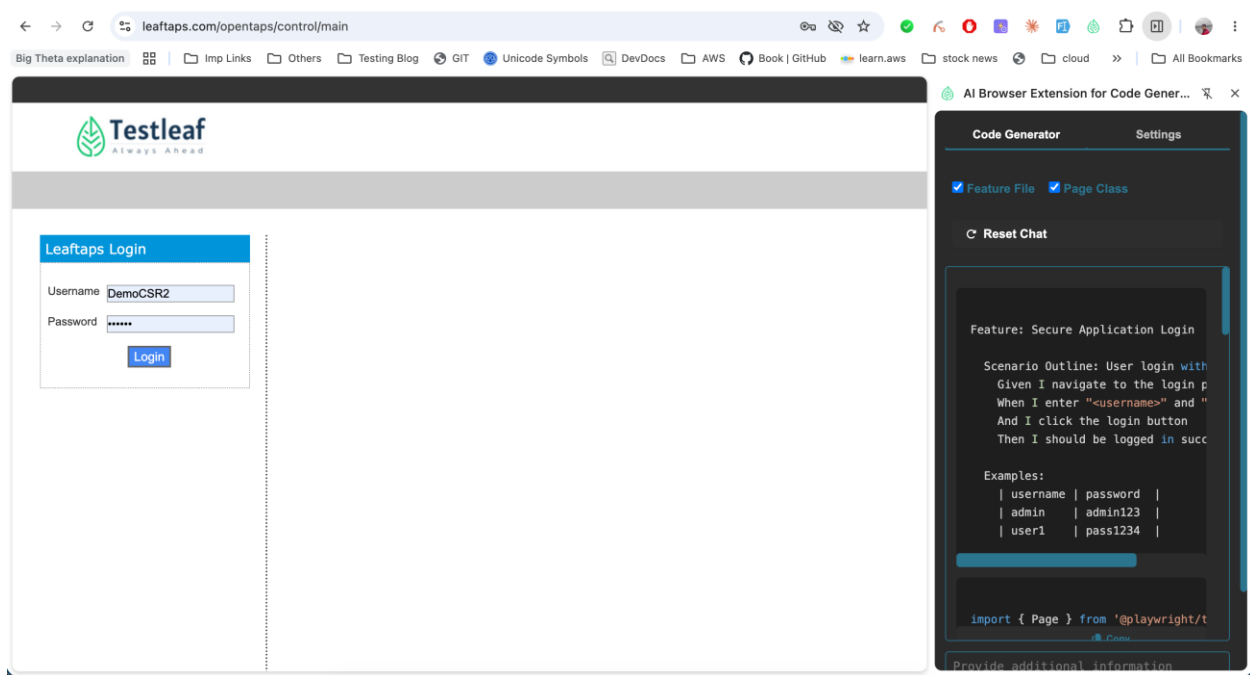
+ |  Agent | Claude Haiku 4.5 | 



After Implementing Typescript and Playwright
Extension



After inspect and Generate
Result



Feature File

Feature: Secure Application Login

Scenario Outline: User login with credentials
Given I navigate to the login page
When I enter "<username>" and "<password>"
And I click the login button
Then I should be logged in successfully

Examples:

	username		password	
	admin		admin123	
	user1		pass1234	

Page Class

Feature: Secure Application Login

Scenario Outline: User login with credentials Given I navigate to the login page When I enter "" and "" And I click the login button Then I should be logged in successfully

Examples: | username | password | | admin | admin123 | | user1 | pass1234 |

```
import { Page } from '@playwright/test';
```

```
/**
```

- Login Page Object Model

```
*/ export class LoginPage { /*
```

- @param page - The Playwright page object

```
*/ constructor(private page: Page) {}
```

```
/**
```

- Navigate to the login page

```
*/ async navigateToLoginPage(): Promise { await
```

```
this.page.goto(https://leaftaps.com/opentaps/control/main);} /*
```

```
/**
```

- Enter the username
- @param username - The username to enter

```
*/ async enterUsername(username: string): Promise { await this.page.locator('input[id="username"]').fill(username); } /*
```

```
/**
```

- Enter the password
- @param password - The password to enter

```
*/ async enterPassword(password: string): Promise { await this.page.locator('input[id="password"]').fill(password); } /*
```

```
/**
```

- Click the login button

```
*/ async clickLoginButton(): Promise { await this.page.locator('input:has-text("Login")').click(); } /*
```

```
/**
```

- Verify if the login was successful
- @returns True if the login was successful, false otherwise

```
*/ async verifyLoginSuccess(): Promise { return this.page.locator('text="Welcome"').isVisible(); } /*
```

```
import { test, expect, Page } from '@playwright/test'; import { LoginPage } from './LoginPage';
```

```

let page: Page; let loginPage: LoginPage;

test.beforeEach(async ({ browser }) => { page = await browser.newPage(); loginPage = new LoginPage(page); });

test.afterEach(async () => { await page.close(); });

test.describe('Secure Application Login', () => { const testData = [{ username: 'admin', password: 'admin123' }, { username: 'user1', password: 'pass1234' }];

testData.forEach(({ username, password }) => { test(User login with credentials - ${username}, async () => { // Given I navigate to the login page await loginPage.navigateToLoginPage();

// When I enter username and password
await loginPage.enterUsername(username);
await loginPage.enterPassword(password);

// And I click the login button
await loginPage.clickLoginButton();

// Then I should be logged in successfully
const isLoggedIn = await loginPage.verifyLoginSuccess();
expect(isLoggedIn).toBeTruthy();
});

}); });

```

Test Pacakage

```

import { test, expect, Page } from '@playwright/test';
import { LoginPage } from './LoginPage';

let page: Page;
let loginPage: LoginPage;

test.beforeEach(async ({ browser }) => {
  page = await browser.newPage();
  loginPage = new LoginPage(page);
});

test.afterEach(async () => {
  await page.close();
});

test.describe('Secure Application Login', () => {
  const testData = [
    { username: 'admin', password: 'admin123' },

```



```

    { username: 'user1', password: 'pass1234' }
  ];

  testData.forEach(({ username, password }) => {
    test(`User login with credentials - ${username}`, async () => {
      // Given I navigate to the login page
      await loginPage.navigateToLoginPage();

      // When I enter username and password
      await loginPage.enterUsername(username);
      await loginPage.enterPassword(password);

      // And I click the login button
      await loginPage.clickLoginButton();

      // Then I should be logged in successfully
      const isLoggedIn = await loginPage.verifyLoginSuccess();
      expect(isLoggedIn).toBeTruthy();
    });
  });
});
});

```

Assignment#02: Go to chat.js Update getPromptKeys() method to handle Playwright/TypeScript combinations

```

getPromptKeys(language, engine) {
  const checkboxes = Array.from(
    document.querySelectorAll('input[name="generationMode"]:checked'),
  );
  const promptKeys = [];
  const lang = language?.toLowerCase() || "";
  const eng = engine?.toLowerCase() || "";

  // Extract selected generation modes (Universal for all languages)
  const isFeatureChecked = checkboxes.some((box) => box.value === "FEATURE");
  const isPageChecked = checkboxes.some((box) => box.value === "PAGE");

  // Validate that at least one option is selected
  if (!isFeatureChecked && !isPageChecked) {
    console.warn(
      "No generation mode selected. Defaulting to Page Object generation.",
    );
    // Default fallback to page object generation
    if (this.isJavaSelenium(lang, eng)) {
      promptKeys.push("SELENIUM_JAVA_PAGE_ONLY");
    } else if (this.isTypeScriptPlaywright(lang, eng)) {
      promptKeys.push("PLAYWRIGHT_TYPESCRIPT_POM");
    }
    return promptKeys;
  }

  // Handle FEATURE + PAGE selections (Universal for all languages/engines)

```

```

if (isFeatureChecked && isPageChecked) {
  // Both feature and page selected - generate combined output
  if (this.isJavaSelenium(lang, eng)) {
    promptKeys.push("CUCUMBER_WITH_SELENIUM_JAVA_STEPS");
  } else if (this.isTypeScriptPlaywright(lang, eng)) {
    promptKeys.push("CUCUMBER_WITH_PLAYWRIGHT_TYPESCRIPT_POM_AND_STEPS");
  } else {
    // For non-Java/Selenium and non-TypeScript/Playwright combinations
    promptKeys.push("CUCUMBER_ONLY");
    this.addUnsupportedLanguageMessage(lang, eng);
  }
} else if (isFeatureChecked) {
  // Feature file only
  promptKeys.push("CUCUMBER_ONLY");
} else if (isPageChecked) {
  // Page object only
  if (this.isJavaSelenium(lang, eng)) {
    promptKeys.push("SELENIUM_JAVA_PAGE_ONLY");
  } else if (this.isTypeScriptPlaywright(lang, eng)) {
    promptKeys.push("PLAYWRIGHT_TYPESCRIPT_POM");
  } else {
    this.addUnsupportedLanguageMessage(lang, eng);
  }
}

return promptKeys;
}

```

Assignment#03: Go to panel.html Add the logics to handle playwright typescript same like selenium Java

```

<!doctype html>
<html>
<head>
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<!-- Fonts, styles, and prism/marked scripts -->
<link
rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.1/css/all.min.css"
/>
<script src="lib/marked/marked.min.js"></script>
<script src="lib/prism/prism.js"></script>
<script src="lib/prism/prism-gherkin-min.js"></script>
<script src="lib/prism/prism-typescript.js"></script>
<link href="lib/prism/prism-tomorrow.css" rel="stylesheet" />
<link href="src/styles/sidepanel.css" rel="stylesheet" />

<!-- Apply appConfig colors immediately to prevent white flash -->
<script src="src/scripts/init-colors.js" type="module"></script>
</head>

<body>
<div class="container">
<!-- Tabs -->
<div class="tabs">
<button class="tab-button active" data-tab="chat">
Code Generator
</button>
<button class="tab-button" data-tab="settings">Settings</button>
</div>

<!-- Code Generator (Chat) Tab -->
<div id="chatTab" class="tab-content active">
<div

```

```

class="generator-options"
style="display: flex; gap: 16px; margin: 8px"
>
<div style="display: flex; flex-direction: column">
<!-- Universal Generation Mode Options -->
<div id="generationModeOptions" style="display: flex; gap: 16px">
<label>
<input
type="checkbox"
name="generationMode"
id="genModeFeature"
value="FEATURE"
checked
/>
<span id="featureLabel">Feature File</span>
</label>
<label>
<input
type="checkbox"
name="generationMode"
id="genModePage"
value="PAGE"
/>
Page Class
</label>
</div>
</div>
</div>
<!-- End generation mode options -->

<div class="controls-section">
<div class="button-container">
<button id="resetChat" class="reset-button">
<i class="fas fa-redo-alt"></i>
<span>Reset Chat</span>
</button>
</div>
</div>

<div class="chat-container">
<div class="chat-messages" id="chatMessages">
<div class="loading-container">
<div class="loading-spinner"></div>
<div class="loading-text">Generating code.</div>
</div>
</div>

<div class="chat-input-container">
<textarea
id="chatInput"
placeholder="Provide additional information"
rows="2"
></textarea>
</div>
<div class="chat-bottom-container">
<div class="chat-input-container">
<button id="inspectButton" class="inspector-toggle">
<i class="fas fa-mouse-pointer"></i>
<span>Inspect</span>
</button>
<button id="sendMessage" class="inspector-toggle">
<i class="fa-solid fa-code"></i>
<span>Generate</span>
</button>
</div>
</div>
</div>
</div>

```

```
<!-- Settings Tab -->
<div id="settingsTab" class="tab-content">
<div class="settings-section">
<!-- Language Binding -->
<div class="model-container">
<label for="languageBinding">Supported Language</label>
<select id="languageBinding">
<option value="java" selected>Java</option>
<option value="csharp">C#</option>
<option value="py">Python</option>
<option value="ts">TypeScript</option>
</select>
</div>
```

```
<!-- Browser Engine -->
<div class="model-container">
<label for="browserEngine">Browser Engine</label>
<select id="browserEngine">
<option value="selenium" selected>Selenium</option>
<option value="playwright">Playwright</option>
<option value="cypress">Cypress</option>
<option value="puppeteer">Puppeteer</option>
</select>
</div>
```

```
<div class="model-container">
<label for="providerSelect">Select LLM Provider</label>
<select id="providerSelect">
<option value="" disabled selected>Choose provider</option>
<option value="groq">Groq</option>
<option value="openai">OpenAI</option>
<option value="claude">Claude</option>
<option value="testleaf">Testleaf</option>
</select>
</div>
```

```
<div class="model-container">
<label for="modelSelect">Select Model</label>
<select id="modelSelect" disabled>
<option value="" disabled selected>Choose model</option>
</select>
</div>
```

```
<div
class="api-key-container"
id="groqKeyContainer"
style="display: none"
>
<label for="groqApiKey">Groq API Key</label>
<input
type="password"
id="groqApiKey"
class="api-key-input"
placeholder="Enter your Groq API key"
/>
</div>
```

```
<div
class="api-key-container"
id="openaiKeyContainer"
style="display: none"
>
<label for="openaiApiKey">OpenAI API Key</label>
<input
type="password"
id="openaiApiKey"
class="api-key-input"
/>
```

```

placeholder="Enter your OpenAI API key"
/>
</div>

<div
class="api-key-container"
id="testleafKeyContainer"
style="display: none"
>
<label for="testleafApiKey">Testleaf API Key</label>
<input
type="password"
id="testleafApiKey"
class="api-key-input"
placeholder="Enter your Testleaf API key"
/>
</div>
</div>
</div>
</div>

<!-- Script references -->
<!-- Initialize app configuration FIRST -->
<script src="src/scripts/init-config.js" type="module"></script>

<script src="src/scripts/popup.js" type="module"></script>
<script src="src/scripts/tabs.js"></script>
<script src="src/scripts/api/groq-api.js"></script>
<script src="src/scripts/api/openai-api.js"></script>
<script src="src/scripts/api/testleaf-api.js"></script>

<!-- Playwright utilities -->
<script src="src/scripts/api/playwright-api.js"></script>
<script src="src/scripts/project-runner.js"></script>
<!-- Chat UI -->
<script src="src/scripts/chat.js" type="module"></script>
</body>
</html>

```